

# PySpark Optimization Scenario Based Question: Solving Small Files Problem

## Question:

### Scenario:

- 100 million records
- Data stored in Parquet format
- 40,000 small files totaling around 90 GB
- Current PySpark job execution time: 14 hours

**Problem:** How can you optimize the PySpark job to reduce runtime?

## Solution with Explanation:

### 1 Problem Identification: Small Files Issue

- Having **40,000 small Parquet files** significantly increases metadata read overhead and shuffle inefficiency. This is commonly known as the **small files problem** in Spark. Target should be fewer, larger files (ideally **256 MB to 1 GB per file**).

### 2 Solution Steps:

#### Step 1: Read Existing Data

```
# Read all existing small parquet filesdf = spark.read.parquet("s3://your-bucket/path/")
```

#### Step 2: Repartition or Coalesce

- Decide number of partitions based on total data size (90 GB):
  - Example: 90 GB / 512 MB = ~180 partitions
- Use **repartition()** for shuffle-based even distribution:

```
df = df.repartition(180) #Target 180 partitions for 90 GB data
```

- Alternatively, use **coalesce()** if reducing partitions without shuffle after narrow transformations:

```
df = df.coalesce(100)
```

#### Step 3: Write Optimized Output

```
df.write.mode("overwrite").parquet("s3://your-bucket/path_optimized/")
```

### 3 Spark Configuration Tuning

SetSpark configurations to optimize file reading and shuffling:

```
spark.conf.set("spark.sql.shuffle.partitions",
200)spark.conf.set("spark.sql.files.maxPartitionBytes", 134217728) # 128
MBspark.conf.set("spark.sql.files.openCostInBytes", 4194304) # 4
MBspark.conf.set("spark.dynamicAllocation.enabled", "true")
```

### 4 Resource Optimization

- Ensure sufficient memory and cores per executor.
- Use dynamic allocation to scale executors up/down based on workload.
- Example EMR / Databricks setup:
  - Executors: 4–8
  - Executor memory: 16–32 GB
  - Executor cores: 4–8

### 5 Result Expectation

| Metric             | Before Optimization | After Optimization |
|--------------------|---------------------|--------------------|
| Small Files Count  | 40,000              | ~180               |
| Total Data Size    | 90 GB               | 90 GB              |
| Job Execution Time | 14 hours            | 1–2 hours          |

### Summary:

- Small files problem was causing metadata and shuffle overhead.
- Repartitioning reduced number of files significantly.
- Optimizing Spark configurations and resource allocation improved performance.

## **Key Takeaway:**

For large datasets stored as numerous small files, always compact them into optimal-sized partitions to avoid performance bottlenecks in Spark jobs.